Data Science and Management 8 (2025) 423–435

Contents lists available at ScienceDirect

Data Science and Management

journal homepage: www.keaipublishing.com/en/journals/data-science-and-management

Research article

# Particle swarm optimization-enhanced machine learning and deep learning techniques for Internet of Things intrusion detection

Mourad Benmalek [a,*], Abdessamed Seddiki [b]

[a] Computer Engineering Department, College of Engineering, Al Yamamah University, Riyadh, 11512, Saudi Arabia
[b] Ecole Nationale Supérieure D'Informatique, BP 68M, Oued-Smar, Algiers, 16309, Algeria

ARTICLE INFO

*Keywords:*
Internet of Things
Intrusion detection system
Particle swarm optimization
Machine learning
Deep learning
Security

ABSTRACT

The exponential escalation of cyber threats and attacks targeting Internet of Things (IoT) devices in this decade necessitates the development of effective intrusion detection methods. This paper presents an innovative anomaly-based intrusion detection system that leverages machine learning (ML) and deep learning (DL) models to secure IoT networks. This study utilizes the RT_IoT2022 dataset, a novel dataset that captures complex IoT attack scenarios. The study implements particle swarm optimization (PSO), a bio-inspired metaheuristic, for feature selection and optimization, successfully reducing computational overhead while enhancing model performance. Several models, including the support vector machine, k-nearest neighbors, categorical boosting (CatBoost), naïve Bayes, convolutional neural network, and long short-term memory, have been evaluated for their ability to classify normal and malicious attacks. Our findings underscore the crucial roles of ML and DL in safeguarding IoT networks and the importance of continuous model evaluation using real-world data. Experiments demonstrate that CatBoost combined with PSO outperforms state-of-the-art methods in the literature on the same dataset across all metrics.

## 1. Introduction

The Internet of Things (IoT) has emerged as a crucial component of the future Internet and has garnered significant interest in recent years (Ma et al., 2019). IoT has permeated various domains, including smart cities (Kyriazis et al., 2013), smart grids (Benmalek et al., 2019), cyber-physical systems (Aguida et al., 2021), healthcare (Faruqui et al., 2021), security (Alaba et al., 2017), supply-chain management (Ben-Daya et al., 2017), blockchain monitoring (Rani et al., 2022), and agriculture (Zhao et al., 2010). Comprising a network of physical smart objects equipped with firmware, software, sensors, and actuators, the IoT enables devices to collect and exchange data with connected devices (Wójcicki et al., 2022). According to Choudhary (2024), the global number of IoT devices is projected to grow from about 15.9 billion in 2023 to over 32.1 billion by 2030. This underscores the critical role of IoT in the near future.

Many devices in IoT networks constantly exchange data over the Internet using various software and communication protocols, potentially exposing vulnerabilities that can be exploited by cyberattacks (Elnakib et al., 2023). The rapid growth in IoT devices and networks has significantly increased these vulnerabilities (Meneghello et al., 2019). Notable attacks include distributed denial of service (DDoS) (Marzano et al., 2018), denial of service (DoS) (Rachmadi et al., 2021), website crippling from botnet attacks (Bahaa et al., 2021), the Mirai botnet (Sharma et al., 2023), and the emerging threat of ransomware, which can encrypt data and demand payment for its release (Benmalek, 2024).

The variety of vulnerabilities in IoT poses a significant challenge. These vulnerabilities can be exploited in multiple ways, making it crucial to address a diverse range of potential attacks on IoT devices (Husnain et al., 2022). Such attacks can compromise or damage a wide range of data types (Al-Garadi et al., 2020), and threaten critical systems that affect our daily lives, including medical devices and environmental monitoring systems (Elnakib et al., 2023). Additionally, the limited computational capabilities and bandwidth capacities of many IoT devices render traditional cybersecurity methods impractical for securing vulnerable environments (Zheng et al., 2022). This highlights the need for specialized approaches, including advanced intrusion detection systems (IDS), to effectively protect IoT systems from cyber threats. IDS, particularly those designed for adaptive network security, provide valuable feedback to network administrators by identifying and

* Corresponding author.
*E-mail address:* m_benmalek@yu.edu.sa (M. Benmalek).
Peer review under responsibility of Xi'an Jiaotong University.

https://doi.org/10.1016/j.dsm.2025.02.005
Received 21 September 2024; Received in revised form 23 February 2025; Accepted 24 February 2025
Available online 28 February 2025

responding to new attack types (Bhavsar et al., 2023). IDS can distinguish between legitimate and malicious network traffic and prevent threats from harming the IoT system (Elnakib et al., 2023).

The widespread adoption of IoT sensors has led to the development of a new approach, anomaly-based IDS, for intrusion detection (Jyothsna et al., 2011). Unlike signature-based IDSs, which rely on known attack patterns, anomaly-based IDSs identify deviations from normal network behavior. This approach involves thorough protocol analysis and rule definition, making it more complex and computationally expensive. However, it also reduces the number of false positives, thereby providing a more accurate detection of potential threats (Jyothsna et al., 2011).

This study aims to develop new approaches for detecting intrusions in IoT networks, given their widespread presence in the digital world. Advances in machine learning (ML) and deep learning (DL) offer promising models that can enhance detection systems, enabling sophisticated network analysis and precise anomaly detection. These models address standard threats and adapt to unforeseen ones. Additionally, the extensive data gathered by IoT sensors provide a rich resource for training and fine-tuning these models, enhancing the effectiveness and accuracy of intrusion-detection mechanisms.

However, existing studies face various gaps, which our study aims to address. Several studies used outdated datasets that do not reflect modern traffic patterns, whereas others did not specifically focus on IoT environments, resulting in a lack of diversity in IoT protocols. Furthermore, prior studies often do not present comprehensive performance metrics, including precision, recall, and F1-score, which are crucial for assessing the effectiveness of IDS in such a critical domain. Additionally, many existing studies have reported low accuracies, undermining their reliability.

Our work addresses these issues by leveraging a novel IoT dataset that contains more modern traffic patterns relevant to IoT infrastructures. Additionally, it provides a comprehensive representation of important performance metrics, achieving very high values across all metrics, thereby demonstrating the effectiveness of our framework.

Our study aims to provide insights and improvements to make a substantial contribution to this field. We utilized the RT_IoT2022 dataset generated by a real-time IoT infrastructure that integrates several IoT devices and employs advanced network attack patterns, providing a robust foundation for developing and testing advanced IDSs. The contributions of our study are summarized as follows.

- We introduce an innovative feature selection method using particle swarm optimization (PSO), a bio-inspired metaheuristic algorithm. By applying PSO to the RT_IoT2022 dataset, we efficiently identify the most relevant features for intrusion detection, resulting in reduced training time and improved model accuracy. The selected feature subset, detailed in Section 3, can serve as a valuable resource for future research on this dataset.
- We apply various ML and DL models to the RT_IoT2022 dataset, a novel and rich dataset that captures complex, real-world IoT attack scenarios. This addresses the gap in existing research that often relies on outdated datasets like UNSW-NB15 and BoT-IoT, thereby enhancing the relevance and applicability of our models.
- We develop an efficient IDS framework that integrates multiple ML and DL methods, providing a comprehensive evaluation of these approaches relative to prior work. By utilizing a broad set of performance metrics, we demonstrate the superior efficiency and accuracy of our proposed IDS, which is capable of detecting anomalies and accurately classifying them into their corresponding attack types.

The remainder of this paper is organized as follows: Section 2 reviews prior works on intrusion detection in IoT systems based on ML and DL models. Section 3 outlines the proposed framework, presenting the data acquisition, data preprocessing, and feature selection, followed by the ML and DL models employed in our study. Section 4 presents the experimental results. Section 5 highlights the managerial implications of the study. Section 6 concludes the paper with the main findings and suggests areas for further research.

## 2. Related work

Recently, IoT intrusion detection has attracted significant research interest. As cyberattacks and detection techniques continue to evolve, an increasing number of research topics have emerged. Consequently, it is crucial to understand the findings and conclusions of previous studies. Researchers have utilized several ML and DL models to develop IDSs by leveraging the datasets generated by IoT devices. Table 1 summarizes the recent studies, highlighting the models, datasets, performance metrics, and identified limitations.

Jan et al. (2019) proposed using a support vector machine (SVM) model with kernels to address attacks that significantly affect IoT network traffic intensity. The linear kernel achieved over 98% accuracy in a short training time; however, this method is limited in its ability to detect low-intensity attacks, which may produce more subtle variations in network traffic.

Yao et al. (2019) proposed the use of LightGBM for lightweight analysis in IoT environments. This model achieved an accuracy of 93.2% and an F1-score of 95.6%, demonstrating its effectiveness in maintaining good performance while optimizing resource use. However, the accuracy and recall, which are critical metrics in intrusion detection, were relatively low. This indicates a potential risk of missing critical threats.

Vitorino et al. (2022) developed an IDS for IoT systems using various ML models and an IoT-23 dataset. They compared supervised (XGBoost, LightGBM, and SVM), unsupervised (local outlier factor (LOF) and iForest), and deep reinforcement learning (DRL) models (DDQN). The supervised models performed best, with LightGBM outperforming in multiclass scenarios. Unsupervised models, although less effective, were effective at detecting rare malware, with iForest showing promise for anomaly detection in smaller datasets. The DRL model demonstrated the potential for continuous learning and adaptation. The main limitation of this study is the inability of the models to distinguish between different classes owing to insufficient training data.

Javaid et al. (2016) introduced a two-level method for IDSs. In their research, the first level involved extracting optimal features using a sparse autoencoder in an unsupervised manner, followed by classification using softmax regression. This method showed promise in enhancing the capability of IDS to identify intrusions and intrusion types. However, this study lacks details on modern evaluation datasets and metrics, limiting the assessment of its effectiveness in real-world IoT deployments.

Kim et al. (2016) proposed an LSTM-RNN model using the KDD Cup 1999 dataset, achieving an accuracy of 87.54%. The approach demonstrated relatively high accuracy. However, the dataset is outdated and does not accurately reflect modern IoT network environments.

Wang et al. (2017) proposed the HAST-IDS framework, which leverages hierarchical spatial-temporal features learned by combining two DL models, convolutional neural networks (CNNs) and long short-term memory (LSTM), aiming to enhance intrusion detection rates. The proposed method was evaluated on two datasets: DARPA1998 and ISCX2012, demonstrating an improved performance in intrusion detection. However, these datasets are somewhat outdated because they do not fully capture modern IoT network environments and contemporary attack types.

Hanif et al. (2019) introduced a shallow artificial neural network as an approach to develop IDS within an IoT environments. The proposed model was tested using the UNSW-NB15 dataset, demonstrating its effectiveness for intrusion detection. However, the study is limited in terms of features related to other IoT protocols, indicating that the data used may not fully represent the diverse IoT traffic.

Kan et al. (2021) introduced an intrusion detection approach using an adaptive PSO-CNN (APSO-CNN). The proposed method involves adjusting the hyperparameters of a one-dimensional CNN using the APSO

**Table 1**
Summary of related works.

| Reference | Model used | Dataset | Metrics | Advantages | Limitations |
|---|---|---|---|---|---|
| Jan et al. (2019) | SVM (linear kernel) | IoT dataset | Accuracy = 98.35% | High accuracy with short training time. | Limitation in attack detection with no effect on traffic intensity. |
| Yao et al. (2019) | LightGBM | IoT environment | Accuracy = 93.2%<br>F1-score = 95.6% | Efficient resource utilization. | Low accuracy and recall, potential risk of missing critical threats. |
| Vitorino et al. (2022) | XGBoost, LightGBM, SVM, LOF, iForest, DDQN | IoT-23 | F1-score = 100% with SVM and LightGBM | Comprehensive comparison of various models for effective intrusion detection. | Lack of data limits the models' ability to distinguish between classes. |
| Javaid et al. (2016) | Sparse autoencoder, softmax regression | NSL-KDD | Accuracy ≥ 98% | Innovative feature extraction method enhances intrusion detection. | Lacks details on modern evaluation datasets and performance metrics. |
| Kim et al. (2016) | LSTM-RNN | KDD Cup 1999 | Accuracy = 87.54% | Achieved relatively high accuracy. | Dataset is outdated. |
| Wang et al. (2017) | HAST-IDS (CNN + LSTM) | DARPA1998, ISCX2012 | Accuracy ≥ 99% for the two datasets | High accuracy achieved through a novel model design. | Datasets are outdated, lacking modern network and attack diversity. |
| Hanif et al. (2019) | ANN | UNSW-NB15 | Accuracy ≥ 97% for binary and multi-class classification | Effective for both binary and multi-class classification tasks. | Limited in terms of features related to other IoT protocols. |
| Kan et al. (2021) | APSO-CNN | N-BaIoT | Accuracy ≥ 95% | Hyperparameter optimization improves performance. | Important evaluation metrics are not provided. |
| Popoola et al. (2021) | Stacking ensemble (RNNs) | BoT-IoT | Achieving 100% in several metrics with binary and multi-class classification. | Effective for binary and multi-class classification tasks. | Dataset has limited attack diversity, and lacks newer and more complex attack types. |
| Sharmila and Nagapadma (2023) | QAE-f16, QAE-u8 (Autoencoder) | RT_IoT2022 | Accuracy = 97.25%<br>Precision = 97.24%<br>Recall = 97.25%<br>F1-score = 97.24% | Reduced memory usage. | Sacrifices performance for memory efficiency. |
| Wu et al. (2024) | FL with attention-based GNN | NSL-KDD | Accuracy = 99.98%<br>Lower F1-score for some attacks (40%) | High accuracy and privacy preservation through FL. | Dataset is outdated, lacking representation of modern IoT threats. |
| Lo et al. (2022) | GNNs | (NF-) BoT-IoT, (NF-) ToN-IoT | Accuracy ≥ 93%, F1-score ≥ 97% for binary classification | Captures network topological features for improved detection. | Limitations in run-time efficiency. |

Note: SVM: support vector machine; IoT: Internet of Things; LightGBM: light gradient boosting machine; XGBoost: eXtreme gradient boosting; LOF: local outlier factor; iForest: isolation forest; DDQN: double deep Q-network; HAST-IDS: hierarchical spatial-temporal features-based intrusion detection system; CNN: convolutional neural network; LSTM: long short-term memory; ANN: artificial neural network; APSO: adaptive particle swarm optimization; RNN: recurrent neural network; FL: federated learning; GNN: graph neural network.

algorithm. Using the N-BaIoT dataset, they evaluated their approach and compared it with three models, achieving the highest values for all metrics. However, several important metrics for evaluating model performance have not yet been provided.

Popoola et al. (2021) proposed a stacking ensemble approach utilizing multiple layers of recurrent neural networks for anomaly detection using imbalanced data gathered from smart home networks using the BoT-IoT dataset. Although their approach demonstrated strong performance, additional validation with different datasets could further reinforce their findings. Additionally, the BoT-IoT dataset has limited attack diversity, and lacks newer and more complex attack types.

Sharmila and Nagapadma (2023) developed an IDS for IoT devices with limited resources. They introduced two optimized autoencoder models, QAE-f16 and QAE-u8, using the RT_IoT2022 dataset, which includes various network traffic types. While the QAE-u8 model showed a significant reduction in memory size and utilization, it did not maintain strong performance in evaluation metrics.

Wu et al. (2024) introduced an IDS based on federated learning (FL) using graph neural networks (GNNs), achieving high accuracy in detecting anomalies using the NSL-KDD dataset. It leveraged FL for privacy protection, whereas the attention mechanisms within the GNN improve detection by capturing complex node interactions. However, the dataset used is outdated, lacking representation of modern IoT threats. In another study, Lo et al. (2022) presented a new approach for detecting intrusions by using a GNN. The proposed model enables the effective detection of attack flows by capturing both the edge features and topological structure of a network flow graph. This method was evaluated using four datasets. It exhibits potential performance, outperforming traditional ML-based classifiers. However, this model has certain limitations, particularly in terms of runtime efficiency. The study of Jiang (2022) also discussed other research efforts that employ GNNs for intrusion detection in IoT environments, outlining the applicability of this approach.

PSO has been applied across various domains, demonstrating its strength in developing solutions for optimization problems. Kumar and Sharma (2018) employed PSO in cloud computing to optimize scheduling, aiming to avoid allocation problems and reduce execution overhead. By adjusting the PSO, their solution effectively reduced the execution time and cost. However, this method has some limitations such as reduced fault tolerance and low scalability. Gul et al. (2021) proposed a meta-heuristic approach by hybridizing PSO with a grey wolf optimizer to solve multi-objective path planning in autonomous guided robots. The approach resulted in path optimization, minimization of distances, and smoothing of routes, overcoming the limitations of conventional path planning techniques.

To enhance fault diagnosis for photovoltaic systems, Eldeghady et al. (2023) combined PSO with a back-propagation neural network (BPNN-PSO). This technique improved convergence rates and prediction accuracy. Additionally, the BP-PSO technique achieved 95% prediction accuracy compared to 87.8% with standard BP.

Notably, no prior research has used the ML and DL models applied in this study on the RT_IoT2022 dataset. Additionally, this study emphasizes several performance metrics to highlight the efficiency of the proposed IDS framework.

## 3. Proposed framework

Here, we present our proposed framework, which is introduced through various key steps, as shown in Fig. 1. First, we describe the data acquisition process, followed by an overview of the data pre-processing

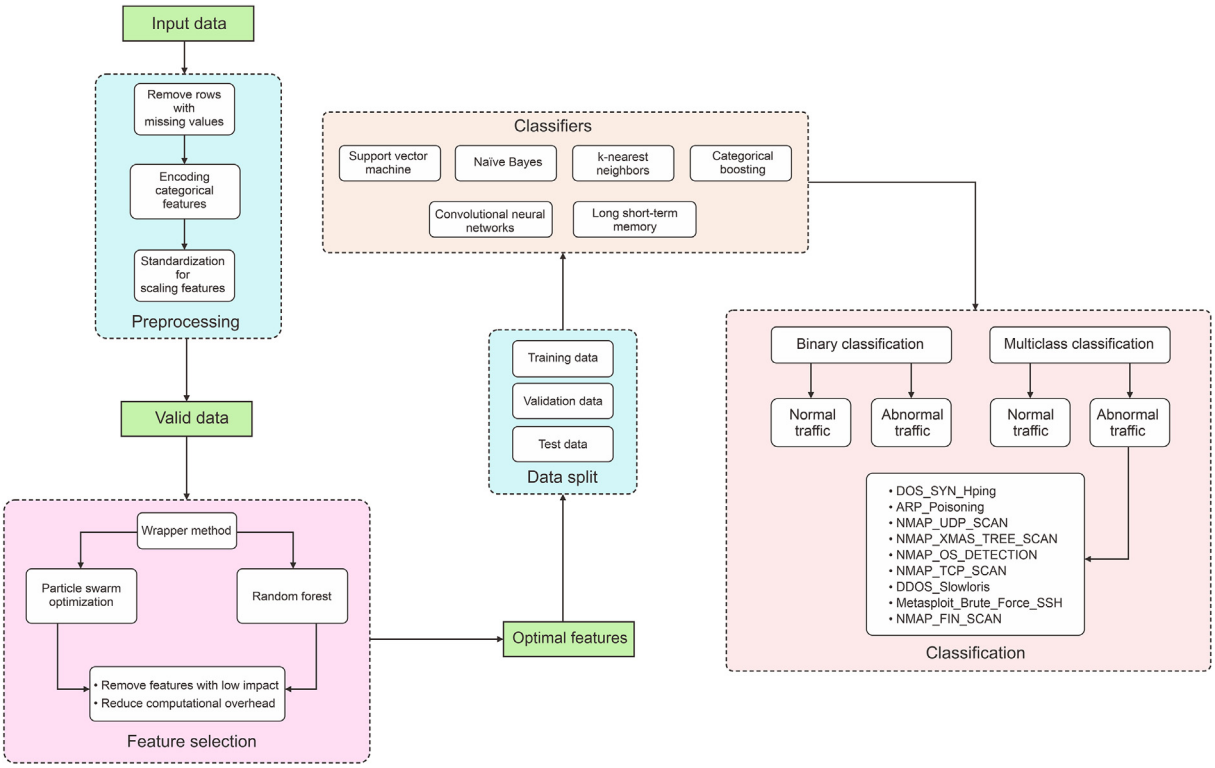


**Fig. 1.** Flowchart of the proposed framework.

techniques employed. Next, we outline the feature selection methodology used to optimize the dataset. Finally, we explore the ML and DL approaches used in this study.

### 3.1. Data acquisition

The RT_IoT2022 dataset is a resource gathered from real-time IoT devices; it is designed to provide a comprehensive representation of both normal and adversarial network behaviors. It was created using a testbed infrastructure composed of an IoT victim and an attacker device connected via a router. Fig. 2 illustrates the infrastructure used to generate the dataset.

Traffic was collected via the router using Wireshark, a network monitoring tool that captures data in PCAP files. The dataset contains four normal profiles and two attack profiles across various IoT devices and configurations.

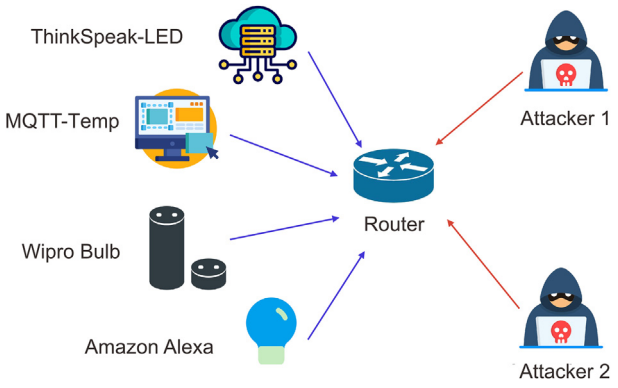


**Fig. 2.** Infrastructure of the RT_IoT2022 dataset (Sharmila and Nagapadma, 2023).

- ThingSpeak-LED: It is an open-source IoT cloud platform dedicated to sensor data visualization and actuator data control. It is used to monitor an RGB LED module connected to an Intel Galileo Gen 2 board.
- MQTT-Temp: A Raspberry Pi with a temperature sensor publish data through an MQTT Mosquitto Broker. MQTT-Temp represents the data samples captured in this traffic.
- Wipro-Bulb: Wipro-Bulb represents the data samples of Wipro's NS9400 Smart Bulb traffic.
- Amazon Alexa: An Alexa device connected to the router, allowing capture of voice-activated cloud interactions.

The RT_IoT2022 dataset offers an extensive description of normal and malicious network activities by incorporating complex network attack techniques. This dataset is considered valuable for training and testing IDSs. The dataset includes data from various IoT devices, capturing both normal and attack patterns.

Despite the limited research on this dataset for IDS implementation, with the notable exception of Sharmila and Nagapadma's (2023) contribution, we identified a valuable opportunity to build on existing studies and explore new approaches for advancing IDS development. To address this gap, we leveraged insights from their research and other related studies to develop a new approach and offer novel solutions for IDS development.

#### 3.1.1. Normal patterns

Table 2 lists the instances of normal patterns and the frequency of each pattern.

**Table 2**
Instances of normal patterns in the RT_IoT2022 dataset.

| Normal patterns | Frequency |
|---|---|
| MQTT | 8,108 |
| ThingSpeak-LED | 4,146 |
| Wipro-Bulb | 253 |

#### 3.1.2. Attack patterns

Table 3 lists instances of various attack patterns in the dataset. The class imbalance in our dataset reflects the real-world distribution of attack types. An attempt to balance the dataset could lead to a representation that does not reflect real-world conditions. By preserving the natural imbalance, our study offers insights into how the model performs in real environments, where rare attacks do not overwhelm the capacity of the model to identify more frequent types.

### 3.2. Experimental setup

The experimental setup consists of a workstation equipped with an Intel Core i7-8650U CPU, 16 GB RAM (main memory), and Intel UHD Graphics. Microsoft Windows 11 Pro was used as the operating system. We trained and tested the models in a Jupyter Notebook environment using Python 3.8.10, with the following libraries: TensorFlow: 2.17.0, Scikit-Learn: 1.5.1, Pandas: 2.2.2, and NumPy: 1.26.4.

### 3.3. Data preprocessing

In this study, various preprocessing steps were applied to prepare the dataset for analysis and model training. First, the dataset was loaded from a CSV file using the pandas library. The first column, which served as an identifier and did not contribute to the predictive modeling, was removed.

Thereafter, rows with missing or incomplete data were identified and removed to ensure the quality and integrity of the dataset. This step is crucial for minimizing bias and improving the accuracy of the model training process.

Categorical encoding was applied to transform nonnumeric categorical variables into a numerical format, making them suitable inputs for ML models. This was achieved using LabelEncoder from the sklearn.-preprocessing module.

Standardization was performed to scale the features, resulting in a mean of 0 and a standard deviation of 1. Standardization was performed using Eq. (1)

$$X_{\text{std}} = \frac{X - \mu}{\sigma} \tag{1}$$

where $X$, $\mu$, $\sigma$, and $X_{\text{std}}$ are the original feature value, mean, standard deviation, and standardized feature value, respectively.

$\mu$ and $\sigma$ are computed using Eqs. (2) and (3), respectively.

$$\mu = \frac{1}{N}\sum_{i=1}^{N} X_i \tag{2}$$

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(X_i - \mu)^2} \tag{3}$$

where $N$, $X_i$, $\mu$, and $\sigma$ denote the number of samples, value of the $i^{th}$ sample, mean, and standard deviation, respectively.

**Table 3**
Instances of attack patterns in the RT_IoT2022 dataset.

| Attack patterns | Instances |
|---|---|
| NMAP_FIN_SCAN | 28 |
| Metasploit_Brute_Force_SSH | 37 |
| DDOS_Slowloris | 534 |
| NMAP_TCP_SCAN | 1,002 |
| NMAP_OS_DETECTION | 2,000 |
| NMAP_XMAS_TREE_SCAN | 2,010 |
| NMAP_UDP_SCAN | 2,590 |
| ARP_Poisoning | 7,750 |
| DOS_SYN_Hping | 94,659 |

The dataset was then divided into input variables ($X$) and output variable ($y$). The "Attack_type" column, representing the target variable indicating different attack types, was separated from the remaining features.

### 3.4. Feature selection

Feature selection significantly impacts the prediction accuracy, which makes it essential before training the ML and DL models (Alhowaide et al., 2020). It aims to enhance the model performance by reducing the number of input variables. This process improves the computational efficiency and reduces overfitting. Three main feature selection methods exist.

(1) Filter methods: These methods evaluate the importance of features using statistical techniques, such as Chi-square tests and correlation coefficients, without relying on any ML model.
(2) Wrapper methods: These methods select feature subsets using an ML model and assess its performance. Examples include forward selection, backward elimination, and recursive feature elimination.
(3) Embedded methods: These methods perform feature selection during the training of the model. Examples include lasso regression (LR), decision trees (DT), and random forests (RF), which perform feature selection as part of their learning algorithms.

In this study, we employ a wrapper method combining the PSO algorithm and RF model. It involves evaluating feature subsets by training a model and assessing its performance.

PSO is an algorithm influenced by the habit of bird flocking or fish schooling (Baruah et al., 2020). Its goal is to find exact or close-to-optimal solutions to NP-hard problems in a search space by simulating the movement of particles. In PSO, each particle symbolizes a potential solution, and its position and velocity are adjusted iteratively according to its own experiences and those of adjacent particles.

In PSO, each particle, $i$, in a $D$-dimensional space is represented by its position, $\boldsymbol{x}_i$, (Eq. (4)) and velocity, $\boldsymbol{v}_i$, (Eq. (5)).

$$\boldsymbol{x}_i = [x_{i1}, x_{i2}, \ldots, x_{iD}] \tag{4}$$

$$\boldsymbol{v}_i = [v_{i1}, v_{i2}, \ldots, v_{iD}] \tag{5}$$

We update the velocity for particle $i$ according to Eq. (6):

$$\boldsymbol{v}_i^{t+1} = \omega \boldsymbol{v}_i^t + c_1 r_1 \left(\boldsymbol{p}_i - \boldsymbol{x}_i^t\right) + c_2 r_2 \left(\boldsymbol{g} - \boldsymbol{x}_i^t\right) \tag{6}$$

where $\boldsymbol{v}_i^{t+1}$ denotes the updated velocity of particle $i$ at iteration $t+1$; $\omega$ denotes an inertia weight allowing to control the influence of the previous velocity; $c_1$ and $c_2$ denote the acceleration coefficients; $r_1$ and $r_2$ denote the random values drawn from a uniform distribution over [0, 1]; $\boldsymbol{p}_i$ denotes the personal best position found by particle $i$; $\boldsymbol{g}$ denotes the global best position discovered by the swarm; and $\boldsymbol{x}_i^t$ is the current position of particle $i$ achieved at iteration $t$.

The position of each particle is updated according to Eq. (7):

$$\boldsymbol{x}_i^{t+1} = \boldsymbol{x}_i^t + \boldsymbol{v}_i^{t+1} \tag{7}$$

The process stops when a predefined number of iterations is achieved, and the selected solution is the last value of particle $\boldsymbol{x}_i$.

RF is an ML model that combines multiple decision-based trees aiming to make predictions. A tree was constructed using a subset of the data and a randomly selected set of features. Using this approach, the likelihood of overfitting is minimized, and the accuracy is improved.

Combining the two methods results in a hybrid process, allowing us to select the following features:

(1) Initialization: Initialize a population of particles, where each particle is a potential solution. A particle in this case is a subset of features selected randomly from the total set. Each subset is represented as a binary vector, where each bit indicates the inclusion or exclusion of a feature.

Particle: A vector (Eq. (4)) where $\boldsymbol{x}_{ij} = 1$ if feature $j$ is included in particle $i$, and $\boldsymbol{x}_{ij} = 0$ otherwise. The vector is initialized randomly.

Velocity: Velocity $\boldsymbol{v}_i$ of particle $\boldsymbol{i}$ determines the direction and magnitude of the change in the particle's position (feature subset) within the search space. This velocity is mathematically represented as shown in Eq. (5).

(2) Evaluation: Train an RF model on the features represented by each particle and evaluate its performance using the accuracy metric. Fitness function $f(\boldsymbol{x}_i)$ is defined as:

$$f(\boldsymbol{x}_i) = \text{Accuracy}(\text{RF trained on } \boldsymbol{x}_i) \tag{8}$$

(3) Update: Apply the PSO update rules to adjust the positions and velocities of the particle using the accuracy of the RF model. We update the velocity and position using Eqs. (6) and (7), respectively.

(4) Selection: Identify the feature subset that yields the highest performance determined by the RF model. The feature subset represented by global best position $\boldsymbol{g}$ is selected for further model training.

As illustrated in Fig. 3, the PSO algorithm effectively improves the accuracy of the model throughout the iterations, demonstrating its potential as a feature selection approach in this context. To develop a model with even higher accuracy, the subset of features selected are used as the input for our training methods. The optimal selected features are listed in Table 4.

### 3.5. ML and DL models

#### 3.5.1. SVM

SVM is a versatile classifier that can be adapted to both binary and multi-classification scenarios. It aims to separate instances with a hyperplane characterized by the Eq. $\boldsymbol{w}^T\boldsymbol{x} + b = 0$, where $\boldsymbol{w}$ is a dimensional coefficient weight vector orthogonal to the hyperplane, $b$ is a bias representing the offset value from the origin, and $\boldsymbol{x}$ is our data points. Applying the SVM aims to determine $w$ and $b$.

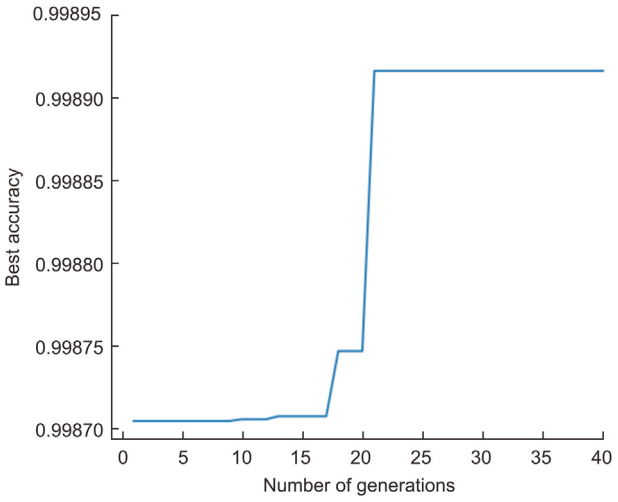


**Fig. 3.** Evolution of model accuracy during feature selection.

**Table 4**
Selected features.

| Feature | Feature |
|---|---|
| down_up_ratio | bwd_iat.std |
| fwd_header_size_min | id.orig_p |
| fwd_iat.std | flow_iat.avg |
| proto | id.resp_p |
| bwd_data_pkts_tot | flow_pkts_payload.min |
| bwd_pkts_tot | fwd_iat.min |
| active.min | fwd_pkts_payload.min |
| bwd_header_size_min | service |
| fwd_iat.avg | bwd_iat.min |
| fwd_bulk_rate | bwd_pkts_payload.max |
| fwd_pkts_payload.tot | fwd_last_window_size |
| fwd_subflow_bytes | fwd_data_pkts_tot |
| bwd_URG_flag_count | fwd_header_size_max |
| Attack_type | bwd_data_pkts_tot |
| bwd_pkts_payload.std | fwd_bulk_bytes |
| idle.max | down_up_ratio |
| bwd_subflow_pkts | bwd_bulk_rate |
| flow_pkts_payload.max | fwd_header_size_tot |
| bwd_iat.avg | fwd_init_window_size |
| bwd_bulk_bytes | bwd_iat.avg |

#### 3.5.2. Naïve Bayes (NB)

NB is a probabilistic classifier. It relies on the Bayes' theorem, assuming feature independence. It uses maximum likelihood estimation to assign each instance to the class with the highest probability.

Mathematically, the NB can be described using the Bayes' theorem, as demonstrated in Eq. (9):

$$P(C_k|x_1, x_2, \ldots, x_n) = \frac{P(C_k) \cdot P(x_1, x_2, \ldots, x_n|C_k)}{P(x_1, x_2, \ldots, x_n)} \tag{9}$$

where $P(C_k|x_1, x_2, \ldots, x_n)$ represents the posterior probability of class $C_k$ given features $x_1, x_2, \ldots, x_n$. $P(C_k)$ represents the prior probability of the class, and $P(x_1, x_2, \ldots, x_n|C_k)$ denotes the likelihood of the features given the class.

#### 3.5.3. k-nearest neighbor (KNN)

KNN is classified as a supervised ML algorithm. It can be adapted for classification purposes. The algorithm makes a prediction after identifying the $k$-nearest data points to an instance and determines the majority class in the case of classification.

Mathematically, a prediction for instance $x$ is made by locating the $k$ closest training examples. The distance between instances is typically measured using metrics, such as the Euclidean distance:

$$d(x, x') = \sqrt{\sum_{i=1}^{n} (x_i - x_i')^2} \tag{10}$$

where $d(x, x')$ represents the Euclidean distance between two instances $x$ and $x'$; and $x_i$ and $x_i'$ represent the feature values of $x$ and $x'$, respectively.

#### 3.5.4. Categorical boosting (CatBoost)

CatBoost is a gradient boosting algorithm that efficiently handles categorical features by transforming them into numerical values. It is also effective for large datasets and complex data structures. It builds on the gradient boosting framework, where models are trained sequentially to correct the errors of previous models with the aim of minimizing a loss function through iterative updates. The general form of the gradient boosting objective function is given as:

$$\mathcal{L} = \sum_{i=1}^{N} \ell(y_i, \hat{y}_i) \tag{11}$$

where $\ell$, $y_i$, and $\hat{y}_i$ represent the loss function, actual labels, and predicted values, respectively.

CatBoost effectively uses categorical features by applying a specific encoding method that involves calculating the statistics of the categorical feature values.

#### 3.5.5. CNNs

Although CNNs were initially created to address challenges in computer vision, they have proven to be an efficient solution for developing IDS, as highlighted by several studies. In this study, CNNs were used owing to their ability to handle large datasets and provide accurate predictions. Because the RT_IoT2022 dataset is one-dimensional, a one-dimensional (1D) CNN was used.

The CNN model implemented in this study consists of multiple layers designed to extract and process input data. The architecture of the CNN is outlined below.

- Input layer: To add an extra-dimension, the input data is reshaped. This process transforms original shape ($n$, $m$) into ($n$, $m$, 1), where $n$ and $m$ represent the number of samples and features, respectively.
- First convolutional layer: It is a 1D layer that has a specified kernel size, a defined number of filters, and an activation function.
- Max pooling layer: A max pooling layer has a defined pool size, which downsamples the input by taking the maximum value over a fixed-size window.
- Flatten layer: It aims to reshape the input, transforming the multidimensional array into a 1D vector.
- Dense layer: This layer is dense and fully connected. It consists of a specified number of units and an activation function, usually the rectified linear unit (ReLU). The ReLU function equation is as follows:

$$\sigma(x) = \max(0, x) \tag{12}$$

- Output layer: A dense layer containing units corresponding to each class, and using the softmax function as an activation function (Eq. (13)) to produce a probability distribution over the classes.

$$\text{softmax}(x) = \frac{e^x}{\sum_j e^{x_j}} \tag{13}$$

#### 3.5.6. LSTM

The LSTM is a common DL model consisting of multiple layers that capture temporal dependencies and process sequential data. Its architecture is as follows.

- Input layer: To add an extra-dimension, the input data is reshaped. This process transforms original shape ($n$, $m$) into ($n$, $m$, 1), where $n$ and $m$ represents the number of samples and features, respectively.
- First LSTM layer: This layer has a specified number of units, and returns the full sequence of outputs, which is used as an input for the next layer.
- Second LSTM layer: The second LSTM layer with a specified number of units, and returns only the last output in the output sequence.
- Dense layer: This layer is dense and fully connected. It consists of a specified number of units and an activation function, usually the rectified linear unit (ReLU) (Eq. (12)).
- Output layer: A dense layer containing units corresponding to each class, using the softmax activation function to produce a probability distribution across all classes.

## 4. Experimental results

Here, we present the experimental results of our proposed intrusion detection framework. We evaluated several ML and DL models on the RT_IoT2022 dataset to assess their performance in both binary and multiclass classification tasks. The results demonstrate the effectiveness of the proposed approach in detecting and classifying IoT network intrusions.

### 4.1. Model hyperparameters

For each evaluated model, specific hyperparameters were selected based on preliminary testing to ensure good performance. Table 5 summarizes the hyperparameters of each model.

### 4.2. Performance metrics

Several performance metrics were used to evaluate the effectiveness of the ML and DL models. These metrics were derived from the confusion matrix (CM). CM summarizes the results of a classification scenario that provides true positives (TP), true negatives (TN), false negatives (FN), and false positives (FP). Their definitions are as follows:

(1) TP: The number of samples accurately identified as being part of a certain class.
(2) FP: The number of samples inaccurately identified as being part of a certain class.
(3) TN: The number of samples accurately identified as not being part of a certain class.
(4) FN: The number of samples accurately identified as not being part of a certain class.

#### 4.2.1. Precision

This metric evaluates the ratio of TP prediction to the total number of samples predicted as positive. Its mathematical formula is as follows:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{14}$$

#### 4.2.2. Recall

Recall assesses the ratio of TP instances to all actual positive cases. It is calculated according to the following equation:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{15}$$

#### 4.2.3. F1-score

It represents the harmonic mean of precision and recall, balancing the tradeoff between precision and recall using a unified metric. Its mathematical formula is as follows:

**Table 5**
Model's hyperparameters.

| Model | Hyperparameters |
|---|---|
| KNN | Number of neighbors ($k$) = 5 |
| SVM | Max iterations = 10,000 |
| CatBoost | Learning rate = 0.1 |
| | Depth = 6 |
| | Iterations = 1,000 |
| | Evaluation metric = MultiClass/Logloss |
| NB | Distribution assumption = Gaussian |
| CNN | Conv layer: 32 filters, kernel size = 3, ReLU activation |
| | Max pooling: pool size = 2 |
| | Dense layer 1: 128 neurons, ReLU activation |
| | Dense layer 2: 64 neurons, ReLU activation |
| | Output layer: sigmoid/softmax activation |
| | Batch size = 32 |
| | Epochs = 10 |
| LSTM | Layer 1: 50 units |
| | Layer 2: 50 units |
| | Dense layer: 64 neurons with ReLU activation |
| | Output layer: softmax activation |
| | Batch size = 32 |
| | Epochs = 10 |

Note: KNN: k-nearest neighbor; SVM: support vector machine; CatBoost: categorical boosting; NB: naïve Bayes; CNN: convolutional neural network; LSTM: long short-term memory; ReLU: rectified linear unit.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (16)$$

#### 4.2.4. Accuracy

Accuracy measures the ratio of instances that are correctly classified (positive and negative) among all instances. It reflects the performance of the model. It is determined according to the following formula:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (17)$$

### 4.3. Results and discussion

This section outlines and examines the results of the experiments conducted on the RT_IoT2022 dataset using two classification approaches.

(1) Binary classification: To categorize data into two classes: "attack" or "normal".
(2) Multiclass classification: To perform a detailed multiclass classification to identify specific types of attacks.

#### 4.3.1. Binary classification

Binary classification aims to identify malicious traffic in a network without detecting the attack type. The two classes used as outcomes were labeled as follows: 0 for normal traffic and 1 for abnormal traffic.

Table 6 lists the performance metrics for each model, and Figs. 4–9 illustrate the CMs for the respective models.

CatBoost emerged as the top performer with near-perfect metrics. It achieved 99.85% accuracy for all metrics, suggesting that it is exceptionally effective at both identifying attack traffic and minimizing FP. KNN and CNN also delivered excellent performance, with all metrics around 99.6%. This indicates that the two models are highly effective in this context. SVM and LSTM performed slightly lower but still showed strong metrics around 97%. Finally, NB lagged behind the other models, with an accuracy of 83.04% and an F1-score of 82.74%. However, it showed a high precision of 91.54%, suggesting that it is good at correctly identifying attack traffics.

#### 4.3.2. Multiclass classification

This approach focuses on assigning data points to the corresponding labels. Particularly, in intrusion detection, multiclass classification seeks not only to separate attacks from normal traffic, but also to categorize each type of attack. 0–8 presents adversarial instances (attacks), and 9 illustrates normal instances. Table 7 provides a mapping between the numerical labels and their corresponding attacks, which facilitates the interpretation of the classification results. Table 8 lists the performance metrics for each model, whereas Figs. 10–15 illustrate the CMs for the respective models.

Both the KNN and CatBoost models demonstrate high performance, achieving accuracies of 99.52% and 99.82%, respectively. Both models maintained a high precision and recall, making them reliable choices for multiclass classification in this context. The CNN model also achieved an impressive accuracy of 98.96%. Its high precision and recall indicate its strong ability to correctly classify both normal and attack instances, benefiting from its DL architecture for learning complex patterns in the data. The SVM and LSTM models achieved slightly lower performance, achieving accuracies of 96.17% and 97.82%, respectively. Although they

**Table 6**
Performance metrics for binary classification models.

| Model | Accuracy (%) | Precision (%) | Recall (%) | F1-score (%) |
|---|---|---|---|---|
| KNN | 99.62 | 99.62 | 99.62 | 99.62 |
| SVM | 97.46 | 97.59 | 97.46 | 97.50 |
| CatBoost | 99.85 | 99.85 | 99.85 | 99.85 |
| NB | 83.04 | 91.54 | 83.04 | 82.74 |
| CNN | 99.56 | 99.56 | 99.56 | 99.56 |
| LSTM | 96.82 | 97.47 | 96.82 | 96.98 |

Note: KNN: k-nearest neighbor; SVM: support vector machine; CatBoost: categorical boosting; NB: naïve Bayes; CNN: convolutional neural network; LSTM: long short-term memory.

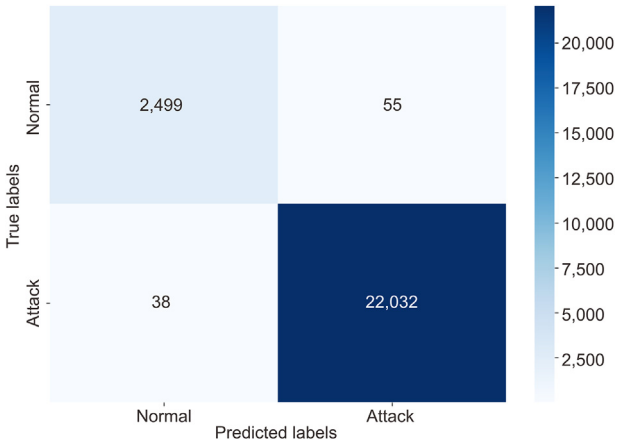


**Fig. 4.** Confusion matrix of the k-nearest neighbor (KNN) for binary classification.

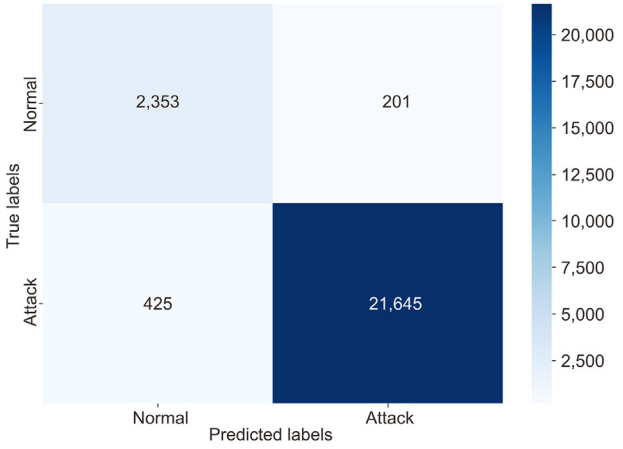


**Fig. 5.** Confusion matrix of the support vector machine (SVM) for binary classification.

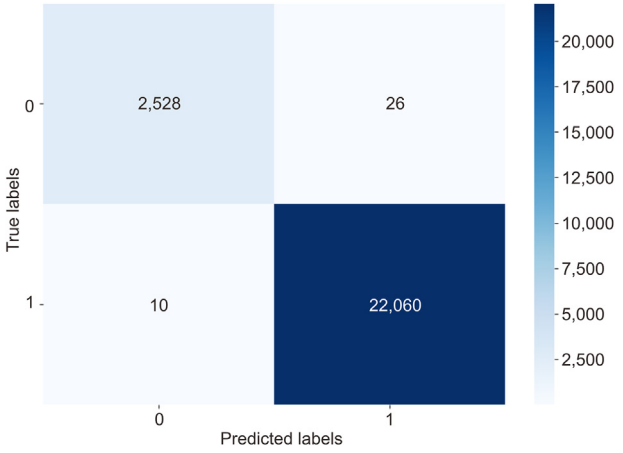


**Fig. 6.** Confusion matrix of the categorical boosting (CatBoost) for binary classification.

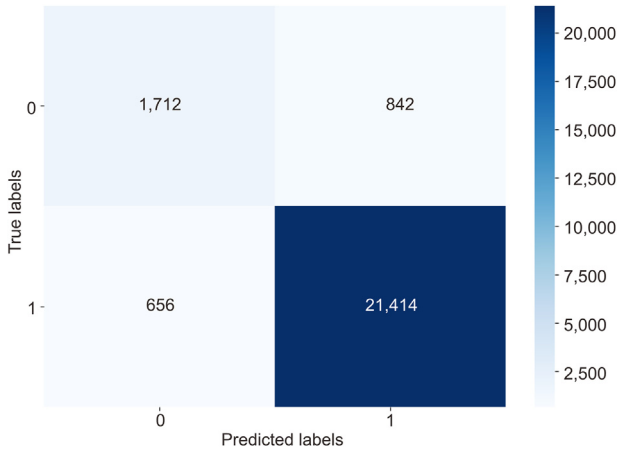


**Fig. 7.** Confusion matrix of naïve Bayes (NB) for binary classification.

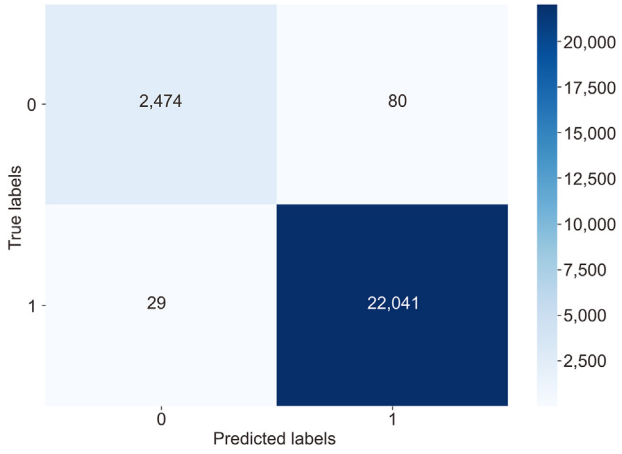


**Fig. 8.** Confusion matrix of the convolutional neural network (CNN) for binary classification.

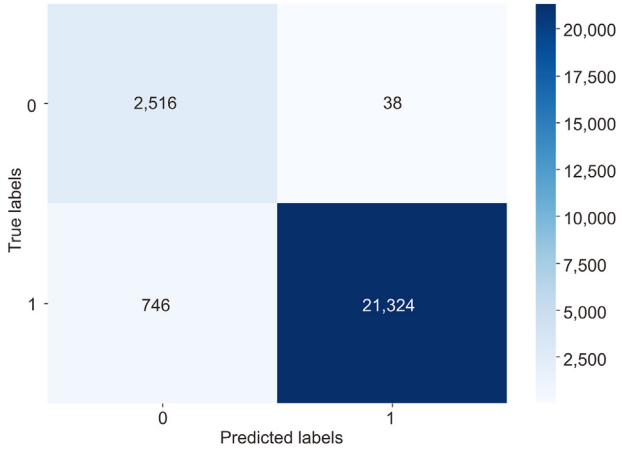


**Fig. 9.** Confusion matrix of the long short-term memory (LSTM) for binary classification.

performed worse than KNN and CatBoost, they still provided strong overall performance with precision and recall close to their accuracy values. For binary classification, NB demonstrated lower metric values, with an accuracy and a recall of 83.04%. This model may struggle with the complexity and imbalance of multiclass datasets, leading to higher FN rates.

**Table 7**
Mapping of attack types to their corresponding label numbers.

| Label number | Attack type |
|---|---|
| 0 | ARP poisioning |
| 1 | DDOS slowloris |
| 2 | DOS SYN hping |
| 3 | Metasploit brute force SSH |
| 4 | NMAP FIN scan |
| 5 | NMAP OS detection |
| 6 | NMAP TCP scan |
| 7 | NMAP UDP scan |
| 8 | NMAP XMAS tree scan |
| 9 | Normal traffic |

**Table 8**
Performance metrics for multiclass classification models.

| Model | Accuracy (%) | Precision (%) | Recall (%) | F1-score (%) |
|---|---|---|---|---|
| KNN | 99.52 | 99.53 | 99.52 | 99.52 |
| SVM | 96.17 | 96.21 | 96.17 | 96.12 |
| CatBoost | 99.82 | 99.82 | 99.82 | 99.82 |
| NB | 83.04 | 91.54 | 83.04 | 82.74 |
| CNN | 98.96 | 98.99 | 98.96 | 98.94 |
| LSTM | 97.82 | 98.07 | 97.82 | 97.72 |

Note: KNN: k-nearest neighbor; SVM: support vector machine; CatBoost: categorical boosting; NB: naïve Bayes; CNN: convolutional neural network; LSTM: long short-term memory.

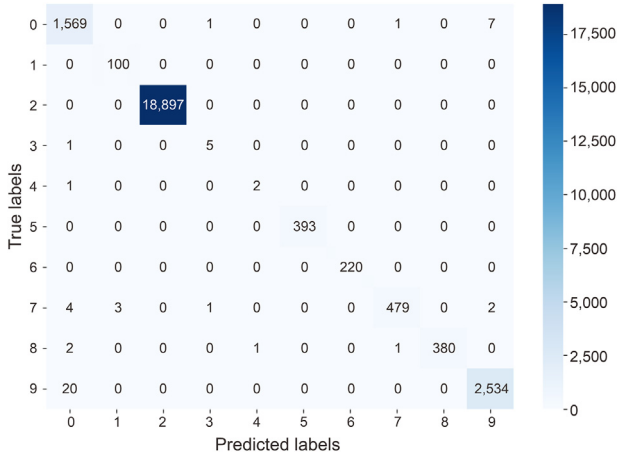


**Fig. 10.** Confusion matrix of the categorical boosting (CatBoost) for multiclass classification.

### 4.3.3. Overall discussion

Evaluating the performance of the model across the two types of classification tasks demonstrates the strengths and weaknesses of each model based on their underlying mathematical principles.

CatBoost is the top-performing model across the two types of classification, with all metrics consistently around 99.85% in binary classification and 99.82% in multiclass classification. This robust performance is primarily attributed to its capability to handle categorical features and its effectiveness in addressing overfitting through ordered boosting. Furthermore, gradient boosting applied to DTs excels in detecting complex patterns in data. Fig. 10 shows that this model rarely misses an attack, thus offering reliable, real-world application for IoT intrusion detection.

KNN also performed well, with metrics around 99.6% in each type of classification. Its simplicity and effectiveness in situations where the data are well-separated make it very strong in this case. It is classified based on its proximity to the nearest neighbors using the Euclidean distance metric demonstrated in Eq. (10), ensuring that samples with similar features are

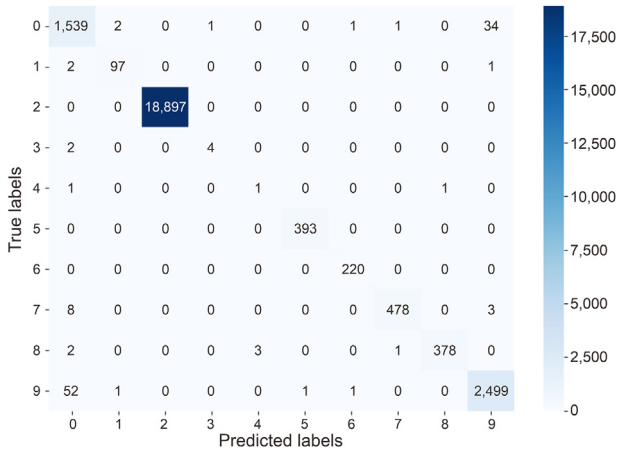


**Fig. 11.** Confusion matrix of the k-nearest neighbor (KNN) for multiclass classification.

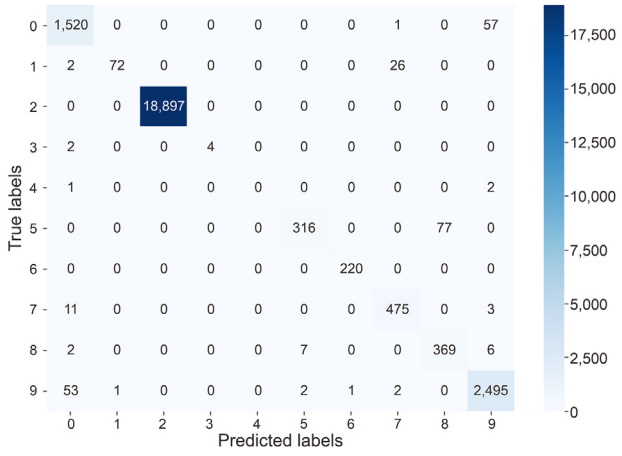


**Fig. 12.** Confusion matrix of the convolutional neural network (CNN) for multiclass classification.

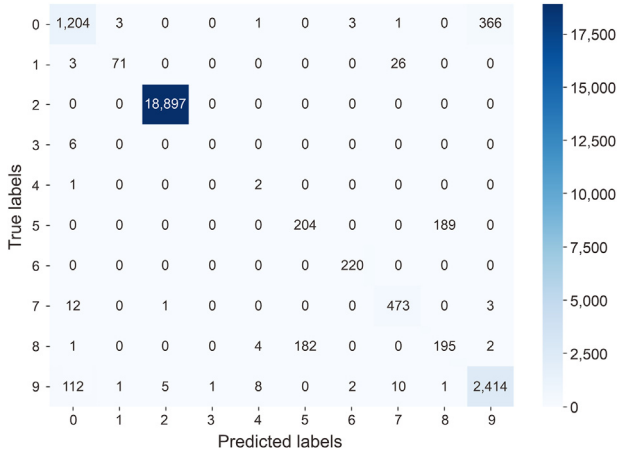


**Fig. 13.** Confusion matrix of the support vector machine (SVM) for multiclass classification.

close together in the feature space. The high accuracy across both tasks indicates that the dataset's feature space was conducive to this type of instance-based learning, providing clear decision boundaries. However, according to Fig. 11, the model is not accurate in classifying certain attack types, such as the NMAP FIN Scan, because of the reduced number

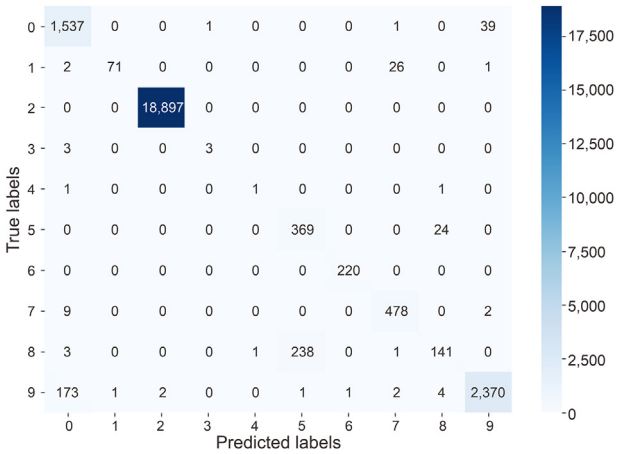


**Fig. 14.** Confusion matrix of the long short-term memory (LSTM) for multiclass classification.

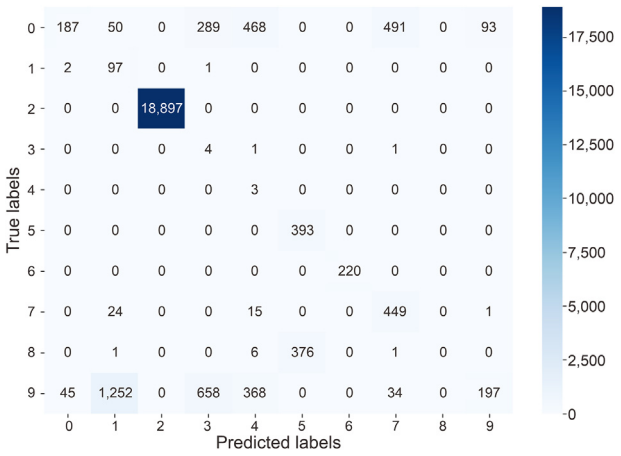


**Fig. 15.** Confusion matrix of the naïve Bayes (NB) for multiclass classification.

of instances of this attack in the dataset. This makes it challenging for the model to predict attacks reliably.

CNN demonstrated strong performance with metrics 99.56% in binary classification and 98.96% in multiclass classification. Its architecture, which is designed to capture spatial hierarchies in data, is particularly suited for tasks that aim to recognize complex patterns. A suitable number of filters and a relatively deep network helped the model to capture more complex patterns. Furthermore, nonlinearity was introduced by choosing the ReLU function (Eq. (12)) as an activation function, allowing the network to learn more complex patterns. Despite its strong overall performance, CNN models have some limitations. As shown in Fig. 12, DDOS Slowloris and NMAP OS Detection were classified with moderate accuracy. However, NMAP FIN Scan anomalies were completely misclassified, reflecting the challenge posed by the low number of instances of this attack in the dataset.

SVM performed well, with metrics 97.46% in binary classification and 96.17% in multiclass classification. The ability of the SVM to create decision boundaries that maximize the margin between classes makes it very strong, particularly in binary classification. However, in the multiclass classification task, SVM can face difficulties while transforming the problem into multiple binary tasks (e.g., one-vs-one or one-vs-rest), leading to an inability to capture the full complexity of the relationships between classes, which justifies the slightly lower accuracy of this type of classification. As shown in Fig. 13, the system particularly struggles with Metasploit Brute Force SSH attacks, which are entirely misclassified. NMAP OS Detection and NMAP XMAS Tree Scan achieved moderate accuracy.

LSTM provided strong results, with 96.98% metrics in binary classification and 97.82% in multiclass classification. LSTM is well-suited for sequential data and tasks involving temporal dependencies, although its performance was slightly lower in this context, likely owing to the absence of temporal patterns in the data. While LSTM performed well overall, as shown in Fig. 14, certain attacks, such as NMAP XMAS Tree Scan, were misclassified, with an accuracy of less than 50%.

NB was the weakest performer model with an accuracy of 83.04% in the two classification approaches. Although it demonstrated a low precision of 91.54%, suggesting that it is good at identifying attack traffic, it struggles with recall and misses some TP. NB assumes that the features are considered conditionally independent given the class, which simplifies likelihood $P(x_1, x_2, \ldots, x_n|C_k)$ into a multiplication of the probabilities of the individual features

$$P(x_1, x_2, \ldots, x_n|C_k) = \prod_{i=1}^{n} P(x_i|C_k) \tag{18}$$

Class prediction using Eq. (18) based on this independence is critical to the simplicity of the model; however, it has some limitations. Real-world data such as network traffic often exhibit complex interdependencies between features. For example, in our network traffic, attributes 'fwd_data_pkts_tot' (Forward Data Packets - Total) and 'wd_header_size_tot' (Forward Header Size - Total) are very correlated because they are related to the amount of data being transmitted in the forward direction (if more packets are being sent, it's often the case that the total header size will also increase proportionally). However, the NB model behaves as if these features are totally independent, which decreases the accuracy of the predictions, resulting in low-performance metrics. Moreover, as shown in Fig. 15, the NB classifier is weak for certain attack types, including ARP Poisoning, NMAP XMAX Tree Scan, and Normal traffic, each with an accuracy of less than 50%.

### 4.4. Impact of PSO-based feature selection on model performance

Table 9 presents the accuracy and training time of various models trained for binary classification with and without PSO-based feature selection.

KNN, SVM, NB, CatBoost, and CNN models reduce processing time with minimal or no loss in accuracy. The reduced computational time is particularly significant in the SVM model, where PSO decreases the runtime from 342.23 s to 14.65 s, i.e., a 23× improvement with only a 1.46% decrease in accuracy. For LSTM, PSO reduces the runtime by 55.56%, with a slight decrease in accuracy.

Notably, IoT devices are constrained by their limited processing power, memory, and energy. Consequently, reducing computational time and resources is crucial for effective IoT deployment. By leveraging PSO, our models achieve efficient operations, making them more suitable for real-time IoT applications.

**Table 9**
Model performance comparison with and without particle swarm optimization (PSO) feature selection for binary classification.

| Model | With PSO | | Without PSO | |
|---|---|---|---|---|
| | Accuracy (%) | Time (s) | Accuracy (%) | Time (s) |
| KNN | 99.62 | 0.01 | 99.66 | 0.02 |
| SVM | 97.46 | 14.65 | 98.92 | 342.23 |
| NB | 93.92 | 0.11 | 94.48 | 0.26 |
| CatBoost | 99.85 | 32.86 | 99.82 | 41.27 |
| CNN | 99.56 | 68.01 | 99.56 | 112.12 |
| LSTM | 96.82 | 691.13 | 98.33 | 1,555.2 |

Note: KNN: k-nearest neighbor; SVM: support vector machine; CatBoost: categorical boosting; NB: naïve Bayes; CNN: convolutional neural network; LSTM: long short-term memory.

### 4.5. Comparison with previous work

This subsection contrasts our findings with those of Sharmila and Nagapadma (2023). Specifically, we compare the results of the CatBoost model from our research with those of the QAE-f16 model examined in their research. CatBoost was selected because it emerged as the best performing classifier in our evaluation, and QAE-f16 was the highest-performing model in the work of Sharmila and Nagapadma (2023).

Fig. 16 illustrates the performance metrics of CatBoost compared with the QAE-f16 model.

Our model (CatBoost) attained an impressive accuracy of 99.85%, surpassing the QAE-f16 model (97.25%). In addition, CatBoost demonstrated notable improvements in the precision, recall, and F1-score, exceeding those of QAE-f16 by approximately 2.6%. These results highlight the effectiveness of our approach in identifying and classifying anomalies and minimizing both FP and FN. In addition, our framework excels not only in detecting anomalies but also in accurately classifying them into their categories. This enhanced capability ensures that anomalies are effectively identified and categorized, thereby ensuring a high level of security for IoT networks.

## 5. Managerial implications

The results of this study have important implications for organizational management of IoT security solutions.

- Enhanced security posture: Implementing the proposed IDS equips organizations with advanced tools to identify and mitigate IoT security threats promptly. This proactive approach reduces the risk of data breaches and cyberattacks, safeguarding organizational assets and customer data.
- Resource optimization: The use of PSO for feature selection reduces computational overhead without compromising accuracy. Managers can allocate resources more efficiently, ensuring that even devices with limited processing capabilities can maintain high levels of security. This leads to cost savings and improved system performance.
- Scalability and adaptability: The IDS framework demonstrates high accuracy across various ML and DL models. Managers can scale the security solutions as the IoT infrastructure expands, adapting to new

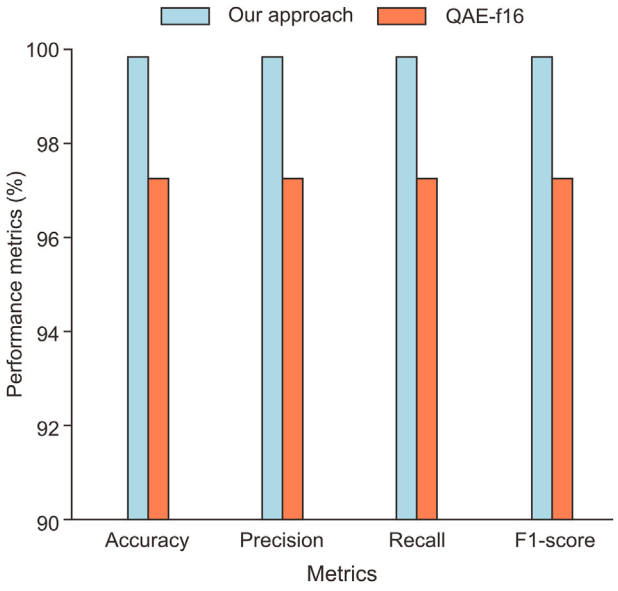


**Fig. 16.** Comparison of performance metrics between our approach and QAE-f16.

threats with minimal disruption. This flexibility is essential for organizations experiencing rapid growth in their IoT ecosystems.

- Regulatory compliance: By enhancing the ability to detect and prevent unauthorized access and data breaches, organizations can better comply with regulatory requirements related to data protection and privacy (e.g., GDPR, HIPAA). This reduces the risk of legal penalties and reputational damage.
- Improved decision-making: The insights gained from advanced anomaly detection can inform strategic decisions regarding network configuration, security policies, and investment in cybersecurity technologies. Managers can make data-driven decisions to strengthen their security frameworks effectively.
- Customer trust and competitive advantage: Demonstrating a strong commitment to IoT security can enhance customer trust and differentiate the organization in the marketplace. It signals to clients and partners that the organization prioritizes cybersecurity, potentially leading to increased business opportunities.

## 6. Conclusion

This study presented a robust anomaly-based IDS for IoT environments using a strong feature selection method based on PSO and various models, including CatBoost, CNN, LSTM, KNN, SVM and NB. The results highlighted the distinct strengths of these models, with CatBoost emerging as the most effective for anomaly detection. It significantly outperformed the QAE-f16 model across all performance metrics, demonstrating its superior capabilities in identifying and classifying anomalies. Moreover, the use of PSO-based feature selection not only improved the accuracy, but it also led to significant reductions in training time. This efficiency is particularly critical in IoT environments, where rapid response times are crucial for anomaly detection.

Future research should focus on leveraging advancements in IoT technology to further enhance model performance. This could involve exploring hybrid approaches that combine different model types, improving data handling techniques, and addressing limitations related to real-time anomaly detection.

## CRediT authorship contribution statement

**Mourad Benmalek:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Methodology, Conceptualization. **Abdessamed Seddiki:** Writing – original draft, Validation, Software, Methodology, Conceptualization.

## Declaration of competing interest

The authors declare that there are no conflicts of interest.

## References


Aguida, M.A., Ouchani, S., Benmalek, M., 2021. An IoT-based framework for an optimal monitoring and control of cyber-physical systems: application on biogas production system. In: 11th International Conference on the Internet of Things. St. Gallen, Switzerland, pp. 143–149.

Alaba, F.A., Othman, M., Hashem, I.A.T., et al., 2017. Internet of Things security: a survey. J. Netw. Comput. Appl. 88 (Jun.), 10–28.

Al-Garadi, M.A., Mohamed, A., Al-Ali, A.K., et al., 2020. A survey of machine and deep learning methods for Internet of Things (IoT) security. IEEE Commun. Surv. Tutor. 22 (3), 1646–1685.

Alhowaide, A., Alsmadi, I., Tang, J., 2020. PCA, random-forest and Pearson correlation for dimensionality reduction in IoT IDS. In: 2020 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS). Vancouver, BC, Canada, pp. 1–6.

Bahaa, A., Abdelaziz, A., Sayed, A., et al., 2021. Monitoring real time security attacks for IoT systems using DevSecOps: a systematic literature review. Information 12 (4), 154.

Baruah, H.S., Thakur, J., Sarmah, S., et al., 2020. A feature selection method using PSO-MI. In: 2020 International Conference on Computational Performance Evaluation (ComPE), Shillong, India, pp. 280–284.

Ben-Daya, M., Hassini, E., Bahroun, Z., 2017. Internet of Things and supply chain management: a literature review. Int. J. Prod. Res. 57 (15–16), 4719–4742.

Benmalek, M., 2024. Ransomware on cyber-physical systems: taxonomies, case studies, security gaps, and open challenges. Internet Things Cyber. Phys. Syst. 4, 186–202.

Benmalek, M., Challal, Y., Derhab, A., 2019. An improved key graph based key management scheme for smart grid AMI systems. In: 2019 IEEE Wireless Communications and Networking Conference (WCNC), Marrakesh, Morocco, pp. 1–6.

Bhavsar, M., Roy, K., Kelly, J., et al., 2023. Anomaly-based intrusion detection system for IoT application. Discov. Internet Things 3 (1), 5.

Choudhary, A., 2024. Internet of Things: a comprehensive overview, architectures, applications, simulation tools, challenges and future directions. Discov. Internet Things 4 (Dec.), 31.

Eldeghady, G.S., Kamal, H.A., Hassan, M.A.M., 2023. Fault diagnosis for PV system using a deep learning optimized via PSO heuristic combination technique. Electr. Eng. 105 (4), 2287–2301.

Elnakib, O., Shaaban, E., Mahmoud, M., et al., 2023. EIDM: deep learning model for IoT intrusion detection systems. J. Supercomput. 79 (12), 13241–13261.

Faruqui, N., Abu Yousuf, M., Whaiduzzaman, M., et al., 2021. LungNet: a hybrid deep-CNN model for lung cancer diagnosis using CT and wearable sensor-based medical IoT data. Comput. Biol. Med. 139 (Dec.), 104961.

Gul, F., Rahiman, W., Alhady, S.S.N., et al., 2021. Meta-heuristic approach for solving multi-objective path planning for autonomous guided robot using PSO–GWO optimization algorithm with evolutionary programming. J. Ambient Intell. Hum. Comput. 12 (7), 7873–7890.

Hanif, S., Ilyas, T., Zeeshan, M., 2019. Intrusion detection in IoT using artificial neural networks on UNSW-15 dataset. In: 2019 IEEE 16th International Conference on Smart Cities: Improving Quality of Life Using ICT & IoT and AI (HONET-ICT), Charlotte, NC, USA, pp. 152–156.

Husnain, M., Hayat, K., Cambiaso, E., et al., 2022. Preventing MQTT vulnerabilities using IoT-enabled intrusion detection system. Sensors 22 (2), 567.

Jan, S.U., Ahmed, S., Shakhov, V., et al., 2019. Toward a lightweight intrusion detection system for the Internet of Things. IEEE Access 7 (Mar.), 42450–42471.

Javaid, A., Niyaz, Q., Sun, W., et al., 2016. A deep learning approach for network intrusion detection system. In: Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS). ACM, New York, pp. 21–26.

Jiang, W., 2022. Graph-based deep learning for communication networks: a survey. Comput. Commun. 185 (Mar.), 40–54.

Jyothsna, V., Rama Prasad, V.V., Munivara Prasad, K., 2011. A review of anomaly based intrusion detection systems. Int. J. Comput. Appl. 28 (7), 26–35.

Kan, X., Fan, Y., Fang, Z., et al., 2021. A novel IoT network intrusion detection approach based on adaptive particle swarm optimization convolutional neural network. Inf. Sci. 568 (Aug.), 147–162.

Kim, J., Kim, J., Thi Thu, H.L., et al., 2016. Long short term memory recurrent neural network classifier for intrusion detection. In: 2016 International Conference on Platform Technology and Service (PlatCon). Jeju, Korea (South), pp. 1–5.

Kumar, M., Sharma, S.C., 2018. PSO-COGENT: cost and energy efficient scheduling in cloud environment with deadline constraint. Sustain. Comput. Inform. Syst. 19 (Sep.), 147–164.

Kyriazis, D., Varvarigou, T., White, D., et al., 2013. Sustainable smart City IoT applications: heat and electricity management & eco-conscious cruise control for public transportation. In: 2013 IEEE 14th International Symposium on "A World of Wireless, Mobile and Multimedia Networks" (WoWMoM). Madrid, Spain, pp. 1–5.

Lo, W.W., Layeghy, S., Sarhan, M., et al., 2022. E-GraphSAGE: a graph neural network based intrusion detection system for IoT. In: NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–9.

Ma, X., Yao, T., Hu, M., et al., 2019. A survey on deep learning empowered IoT applications. IEEE Access 7 (Dec.), 181721–181732.

Marzano, A., Alexander, D., Fonseca, O., et al., 2018. The evolution of bashlite and mirai IoT botnets. In: 2018 IEEE Symposium on Computers and Communications (ISCC), pp. 813–818.

Meneghello, F., Calore, M., Zucchetto, D., et al., 2019. IoT: Internet of threats? A survey of practical security vulnerabilities in real IoT devices. IEEE Internet Things J. 6 (5), 8182–8201.

Popoola, S.I., Adebisi, B., Hammoudeh, M., et al., 2021. Stacked recurrent neural network for botnet detection in smart homes. Comput. Electr. Eng. 92 (Jun.), 107039.

Rachmadi, S., Mandala, S., Oktaria, D., 2021. Detection of DoS attack using AdaBoost algorithm on IoT system. In: International Conference on Data Science and Its Applications (ICoDSA), Bandung, Indonesia, pp. 28–33.

Rani, P., Kaur, P., Jain, V. et al., 2022. Blockchain-based IoT enabled health monitoring system. J. Supercomput. 78 (May), 17284–17308.

Sharma, A., Mansotra, V., Singh, K., 2023. Detection of Mirai botnet attacks on IoT devices using deep learning. J. Sci. Res. Technol. 1 (6), 174–187.

Sharmila, B.S., Nagapadma, R., 2023. Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. Cybersecurity 6 (1), 41.

Vitorino, J., Andrade, R., Praça, I., et al., 2022. A comparative analysis of machine learning techniques for IoT intrusion detection. In: Aïmeur, E., Laurent, M., Yaich, R., et al. (Eds.), Foundations and Practice of Security. FPS 2021. Lecture Notes in Computer Science, vol. 13291. Springer, Cham, pp. 191–207.

Wang, W., Sheng, Y., Wang, J., et al., 2017. HAST-IDS: learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection. IEEE Access 6 (Dec.), 1792–1806.

Wójcicki, K., Biegańska, M., Paliwoda, B., et al., 2022. Internet of Things in industry: research profiling, application, challenges and opportunities: a review. Energies 15 (5), 1806.

Wu, J., Qiu, G., Wu, C., et al., 2024. Federated learning for network attack detection using attention-based graph neural networks. Sci. Rep. 14 (1), 19088.
Yao, H., Gao, P., Zhang, P., et al., 2019. Hybrid intrusion detection system for edge-based IIoT relying on machine-learning-aided detection. IEEE Netw. 33 (5), 75–81.
Zhao, J.C., Zhang, J.F., Feng, Y., et al., 2010. The study and application of the IOT technology in agriculture. In: 2010 3rd International Conference on Computer Science and Information Technology, pp. 462–465.
Zheng, Y., Li, Z., Xu, X., et al., 2022. Dynamic defenses in cyber security: techniques, methods and challenges. Digit. Commun. Netw. 8 (4), 422–435.